

System Architecture Document

Autonomous AI-Powered Building Management System (AI-BMS)

Tool-Calling Architecture · Prompt Engineering · Latency Management · Simulation Log Handling

1. Overview

The AI-BMS is a closed-loop HVAC control system built around an **EnergyPlus digital twin** and a **local Qwen LLM** acting as a supervisory decision-maker. The core design principle is that the LLM is never given direct, unrestricted control of the HVAC actuators — every AI decision must pass through deterministic validation and safety layers before it reaches the building.

Hybrid control philosophy:

LLM Reasoning → Semantic Validation → Deterministic Safety → Real-Time Comfort Protection → Fail-Safe Control

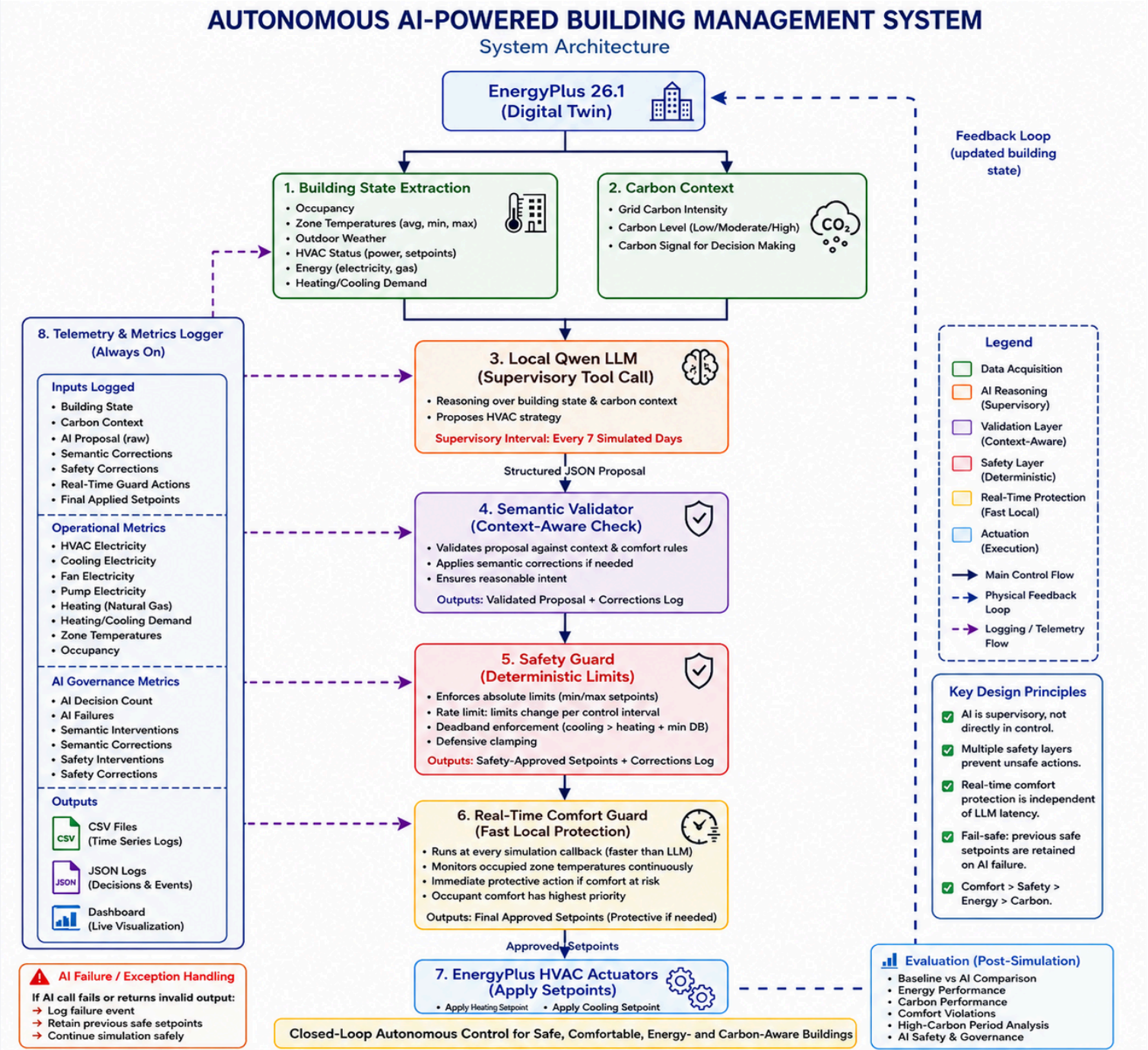


Figure 1. End-to-end closed-loop architecture. Green = data acquisition, orange = AI reasoning (supervisory only), purple = context-aware validation, red = deterministic safety, yellow = fast real-time protection, blue = actuation. Solid arrows are the main control flow, the dashed line on the right is the physical feedback loop back into EnergyPlus, and the dashed purple arrows on the left are the always-on telemetry/logging flow.

The eight labeled stages map directly onto the tool-calling pipeline described below:

1. **Building State Extraction** — occupancy, zone temperatures, outdoor weather, HVAC status, energy, heating/cooling demand
2. **Carbon Context** — grid carbon intensity and carbon level (low/moderate/high)
3. **Local Qwen LLM** — supervisory reasoning, called once every 6 simulated hours
4. **Semantic Validator** — context-aware check against comfort rules, with a corrections log
5. **Safety Guard** — deterministic absolute limits, rate limiting, deadband enforcement, defensive clamping
6. **Real-Time Comfort Guard** — fast local protection running at every simulation callback
7. **EnergyPlus HVAC Actuators** — applies the final approved heating/cooling setpoints
8. **Telemetry & Metrics Logger** — always-on logging of inputs, operational metrics, and AI governance metrics to CSV/JSON, feeding the live dashboard and the post-simulation evaluation

The diagram's key design principles summarize the philosophy behind every section that follows: the AI is supervisory rather than directly in control, multiple safety layers prevent unsafe actions, real-time comfort protection is independent of LLM latency, the system fails safe by retaining the previous setpoints on AI failure, and the priority order is always **Comfort > Safety > Energy > Carbon**.

2. Tool-Calling Architecture

The system treats each major component as a **callable tool** in an orchestration pipeline, coordinated entirely by Python (`simulation/run_ai_control.py`). The LLM is one tool among several, not a freestanding agent.

Tool	Role	Called by
EnergyPlus API	Building physics / digital twin simulation	Python controller, every timestep
Building State Tool	Converts raw EnergyPlus output into a compact structured state (occupancy, temperatures, setpoints, carbon level)	Python controller
Qwen LLM Tool	Supervisory reasoning; returns a structured JSON decision (<code>heating</code> , <code>cooling</code> , <code>reason</code> , <code>energy_strategy</code> , <code>comfort_risk</code>)	Python controller, once per AI interval
Semantic Validator	Rule-based check on whether the LLM's proposal fits the current context	Python controller, after every LLM call
Safety Guard	Deterministic clamps: absolute limits, rate-of-change limits, minimum deadband	Python controller, after semantic validation
Real-Time Comfort Guard	Lightweight local logic that runs every timestep, independent of the LLM	Python controller, continuously
Metrics Logger / Dashboard	Records telemetry and exposes live state	Python controller, in parallel

Why tool-calling instead of direct control: the LLM only ever *proposes* a structured action (stage 3 in Figure 1). It never writes to an EnergyPlus actuator itself. This keeps the LLM's "blast radius" limited to a JSON object that downstream deterministic tools (stages 4–6) can inspect, correct, log, or reject — turning what would otherwise be a single point of failure into an auditable pipeline that always terminates at the always-on Telemetry & Metrics Logger (stage 8).



Figure 2. Each tool in the chain asks a different question of the same proposal — contextual sense, hard limits, then real-time drift — before anything reaches the actuators.

Each downstream tool has a narrow, single responsibility:

- **Semantic Validator** — "Does this decision make sense for the current occupancy/temperature/carbon context?" (e.g., raises heating if a proposal would leave an occupied, cold room under-heated).
- **Safety Guard** — "Is this decision within hard operational bounds, regardless of context?" (absolute setpoint range, max setpoint change per step, minimum heating-cooling deadband).
- **Real-Time Comfort Guard** — "Has the building drifted out of comfort since the last AI call?" Operates independently of the LLM's slower interval so comfort protection is never blocked on inference.

If any tool in the chain fails (LLM crash, malformed JSON, parsing error), the controller falls back to the last known-safe setpoints rather than propagating an unvalidated value — this is the fail-safe path described in Section 6.

3. Prompt Engineering Strategy

Because the LLM is a control-loop component and not a chat interface, prompt design is treated as an engineering constraint, not a UX detail.

1. **Structured, minimal context** — the prompt carries only decision-relevant fields (occupancy, indoor temperature range, outdoor temperature, current setpoints, carbon intensity/level). Raw EnergyPlus output is never passed in; it is pre-compressed by the Building State Tool.
2. **Explicit priority ordering** — the prompt encodes a fixed reasoning hierarchy so the model doesn't have to infer trade-offs:
 1. Safe HVAC operation
 2. Occupant comfort
 3. Avoid unnecessary HVAC activity
 4. Reduce energy consumption
 5. Prefer lower-carbon operation *when comfort allows*
3. **Structured output contract** — the model must return a fixed JSON shape (`heating` , `cooling` , `reason` , `energy_strategy` , `comfort_risk`) rather than free text, so the response can be parsed and validated programmatically without an extra extraction step.
4. **Forced explainability** — requiring `reason` and `energy_strategy` fields means every decision carries its own justification, which both the Semantic Validator's corrections and the evaluation logs can reference later.
5. **Defense-in-depth, not prompt-only safety** — the prompt is not trusted as a safety mechanism by itself. It reduces the *frequency* of bad proposals, but the Semantic Validator and Safety Guard are what actually guarantee correctness, since a well-crafted prompt can still yield an occasional wrong output.

4. Prompt Latency Management

LLM inference is orders of magnitude slower than deterministic Python logic, so the architecture deliberately decouples "how often the building is protected" from "how often the LLM is asked for an opinion."

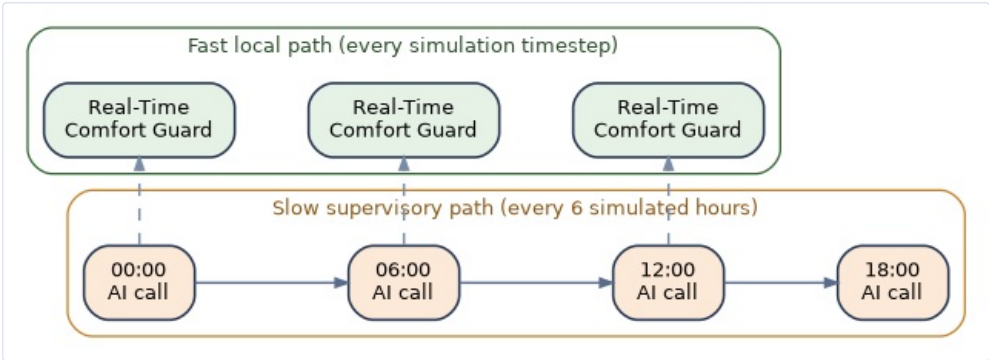


Figure 3. The LLM is consulted on a fixed interval; the Real-Time Comfort Guard runs on every simulation timestep in between, so occupant protection never waits on inference latency.

- **Supervisory AI interval:** the LLM is called once every **6 simulated hours** (`ai_interval_hours: 6`) rather than at every EnergyPlus timestep, which sharply cuts the number of inference calls and total simulation runtime.
- **Interval caching:** the controller tracks the last processed interval and only issues a new LLM request when the simulation clock enters a new interval, preventing duplicate calls triggered by multiple EnergyPlus callbacks within the same window.
- **Local inference:** Qwen runs locally, removing network round-trip latency and external API dependency from the control loop, and keeping building-state data private.
- **Fast local guards fill the gap:** between AI calls, the Real-Time Comfort Guard — lightweight, deterministic Python — reacts immediately to temperature drift, so the six-hour AI cadence never becomes a six-hour blind spot for occupant comfort.

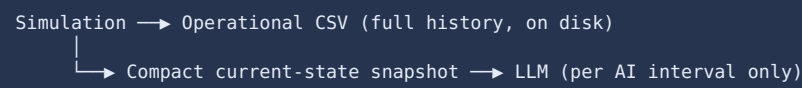
Net effect: expensive reasoning happens rarely and asynchronously from real-time protection, which happens continuously and cheaply.

5. Handling Lengthy Simulation Logs

A full EnergyPlus run produces thousands of timestep records; feeding that history into the LLM would be both wasteful and unnecessary for a single control decision.

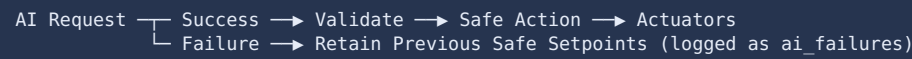
- **Current-state summarization:** the LLM only ever receives a compact snapshot of *now* (current occupancy, indoor/outdoor temperature, current setpoints, current carbon level) — never the historical time series. This acts as a state-compression step between the simulation and the model.
- **CSV-based operational logging:** detailed telemetry is written continuously to disk (`baseline_metrics.csv` , `ai_metrics.csv` , `comparison_summary.csv`) rather than accumulated in LLM context, keeping simulation history and AI prompt memory fully separate.

- **Controlled logging cadence:** operational metrics are logged at tracked intervals (not on every internal callback), avoiding redundant duplicate rows while still preserving a usable time series.
- **Separate baseline vs. AI logs:** the two simulation modes write to independent files; the comparison engine aligns them by matching simulation timestamps after the fact, rather than merging histories during the run.
- **Post-simulation aggregation:** heavier analysis — energy totals, carbon totals, comfort-violation rates, intervention counts — is computed by `evaluation/compare_results.py` *after* the simulation completes, so large historical datasets never need to enter the real-time control loop at all.



This keeps the real-time path lightweight regardless of how long the simulation runs, while still leaving a complete, inspectable record for evaluation.

6. Fail-Safe Behavior



Any exception in the LLM call, response parsing, or unexpected output triggers this fallback rather than applying an unvalidated value — the building always defaults to the last known-safe state. Concretely, on a failed or invalid AI call the controller: logs the failure event, retains the previous setpoints, and continues the simulation safely without interruption.

This same telemetry backbone (stage 8 in Figure 1) is what the post-simulation **Evaluation** stage reads from — comparing the baseline run against the AI-controlled run across energy performance, carbon performance, comfort violations, high-carbon-period behavior, and AI safety/governance metrics.

7. Summary

Concern	Mechanism
Tool orchestration	Python-coordinated pipeline: EnergyPlus → State → LLM → Semantic Validator → Safety Guard → Real-Time Comfort Guard → Actuators
Prompt design	Minimal structured context, explicit priority order, forced structured/explainable output
Latency	6-hour supervisory AI interval + interval caching + local inference + always-on fast guard
Long logs	Current-state compression to the LLM, full history to CSV, post-run aggregation for evaluation
Safety net	Deterministic limits, rate limiting, deadband enforcement, and fail-safe fallback on any AI error

The result is a system where the LLM contributes flexible, explainable reasoning, while every action it proposes is independently verified before it can affect the simulated building.